

Estruturas de dados

Listas encadeadas



Listas encadeadas

- Pré-requisitos:
 - Registros;
 - Ponteiros;
 - Alocação dinâmica de memória.



Lista encadeada

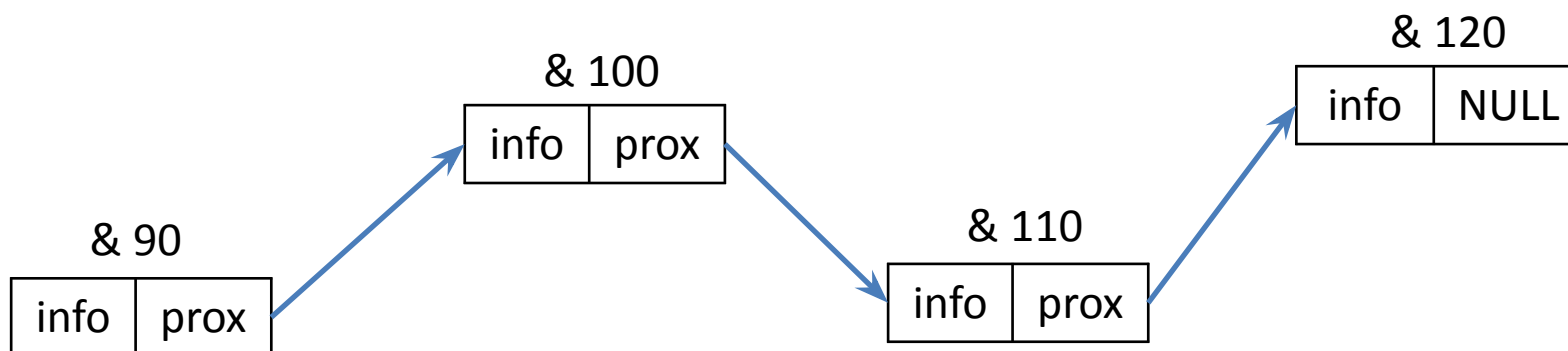
- Refere-se a uma estrutura de dados mais dinâmica que os vetores pois a alocação/liberação de memória é realizada conforme a criação ou retirada de elementos.
 - Assim, não existe uma quantidade predefinida de memória para o total de elementos. Não é necessário saber o total de elementos.
- Composta de **nós (ou nodos)** que guardam informações sobre cada elemento.
- Define-se cada elemento como um registro que possui:
 - campos de informações;
 - ponteiro para o próximo elemento da lista.

& 90

info	prox
(int, float, char, struct, etc)	(endereço de memória do próximo nó)

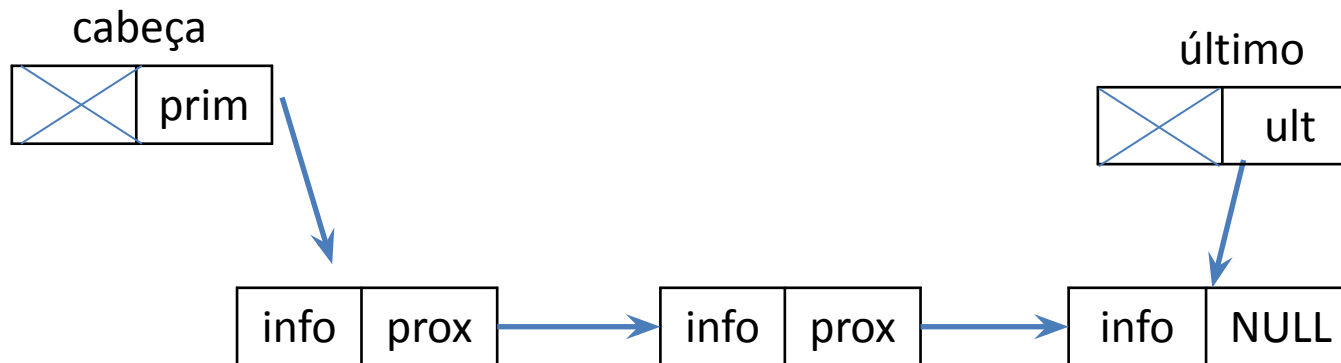
Lista encadeada

- Características:
 - Tamanho da lista não é pré-definido;
 - Espaço utilizado em memória é proporcional ao número de elementos;
 - Elementos ocupam posições não contíguas na memória;
 - O que os caracteriza como um conjunto de elementos (lista)?
 - Cada elemento guarda endereço de memória do próximo.



Lista encadeada

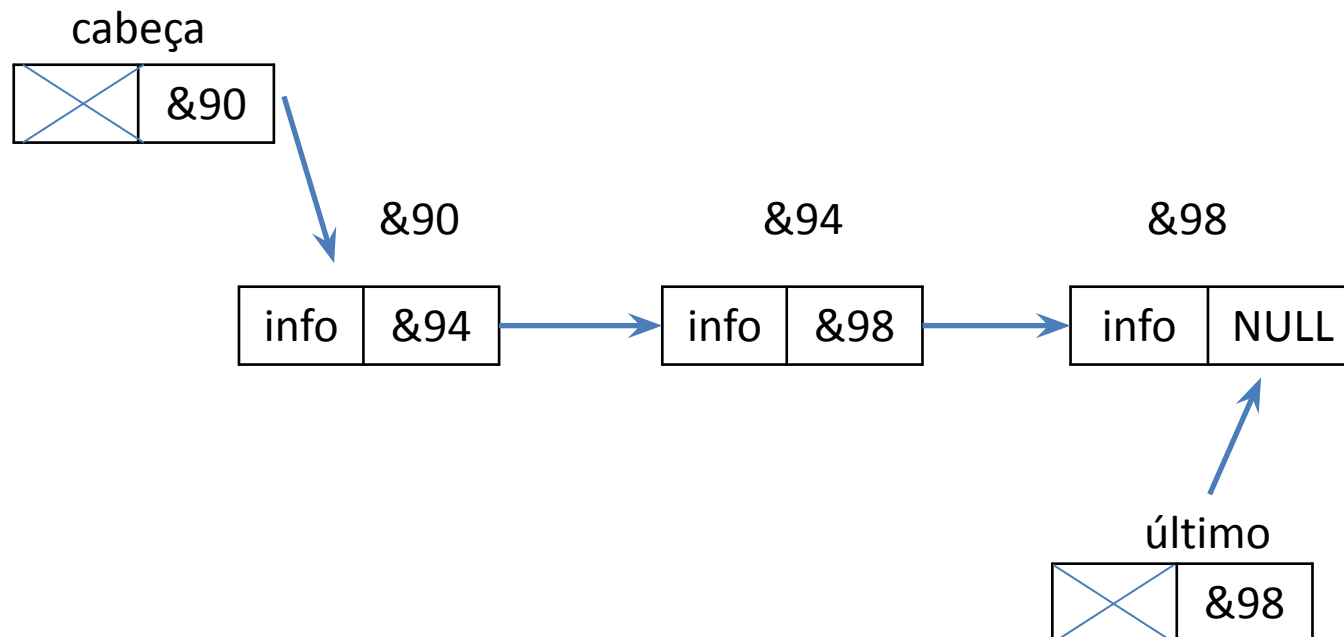
- Deve ter uma célula cabeça:
 - Não guarda informações, apenas o endereço de memória do primeiro nó;



- Pode ter um apontador para o último elemento.

Lista encadeada

- Exemplo:



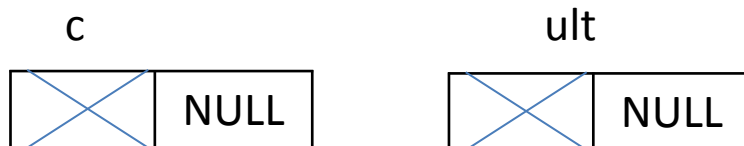
- Acesso aos elementos **não** é feito de forma independente!
- Para acessar o elemento alocado no endereço &98, deve-se passar pelos seus antecessores. Logo, deve-se sempre saber quem está no início da lista.

Lista vazia

- Quando não existem elementos na lista o nó cabeça possui um valor nulo (NULL);
- Ponteiro do último elemento também possui valor nulo (NULL).

`c = NULL;`

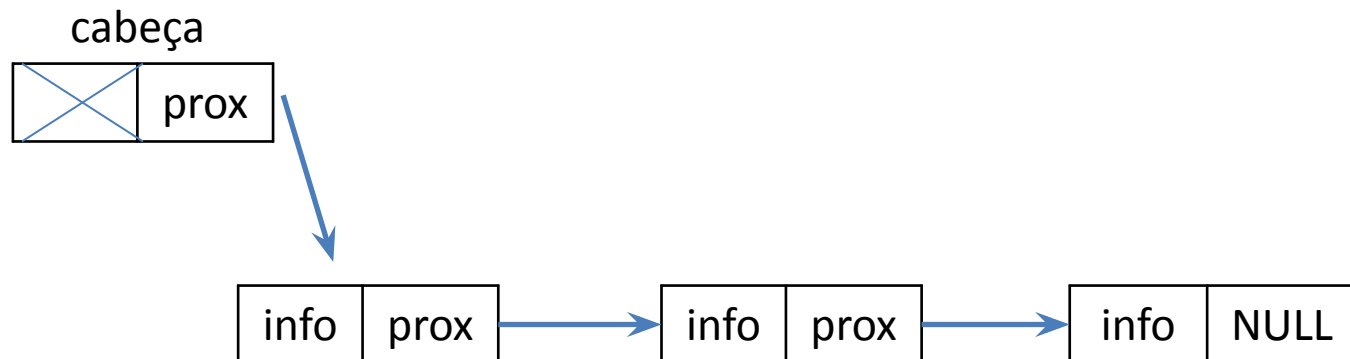
`ult = NULL;`



- Principais manipulações:
 - Inserção de novos elementos;
 - Remoção de elementos existente;
 - Acesso e manipulação de valores dos nós.

Inserir elementos na lista

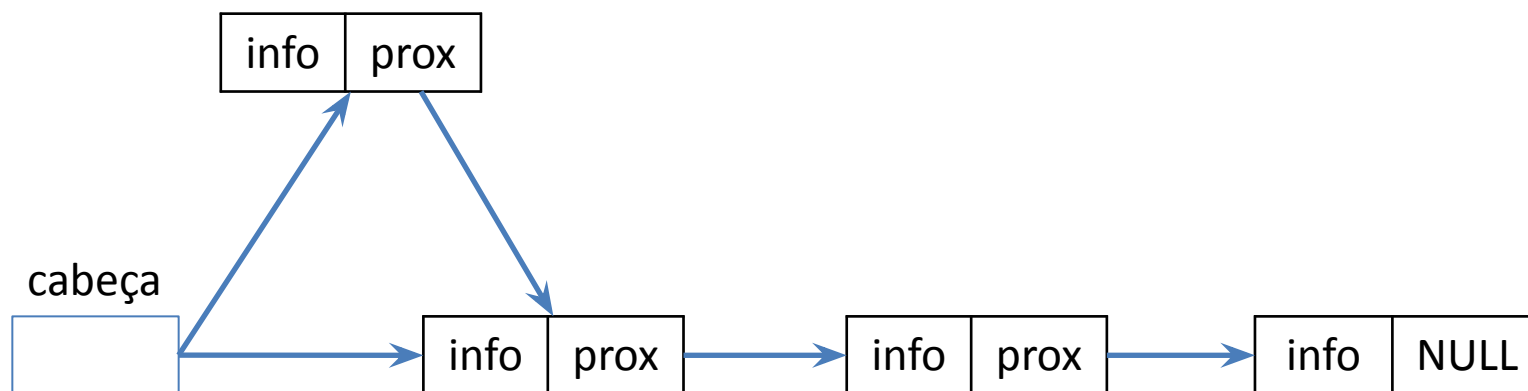
- 3 opções de inserção de elementos:
 - Início da lista: 1ª posição (posição mais à esquerda na imagem abaixo);
 - Final da lista: última posição (posição mais à direita na imagem);
 - Meio da lista: entre dois elementos quaisquer.





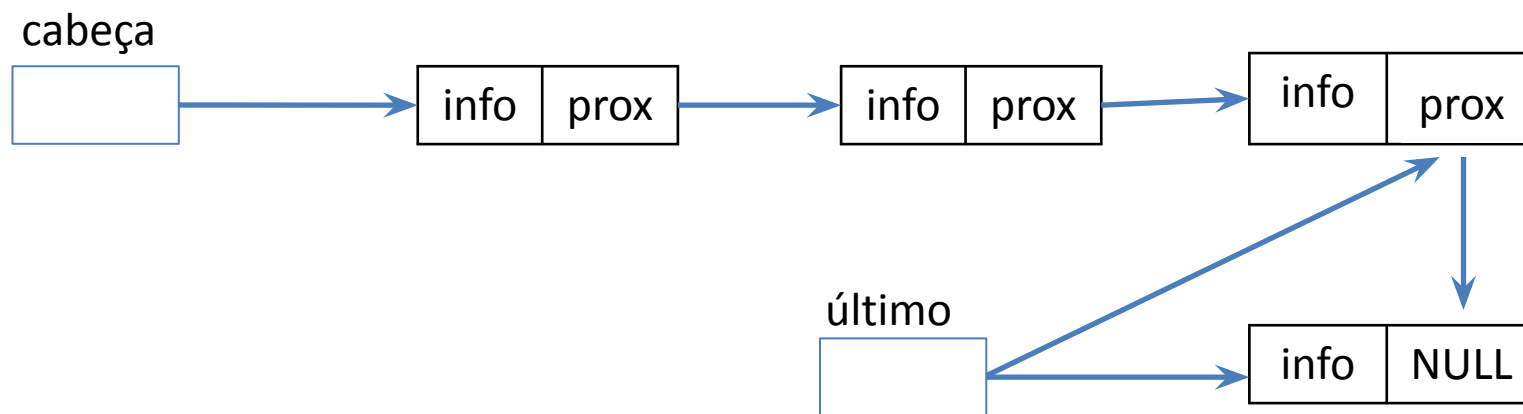
Inserção na 1ª posição

- Nó cabeça **SEMPRE** deve apontar para o primeiro elemento!





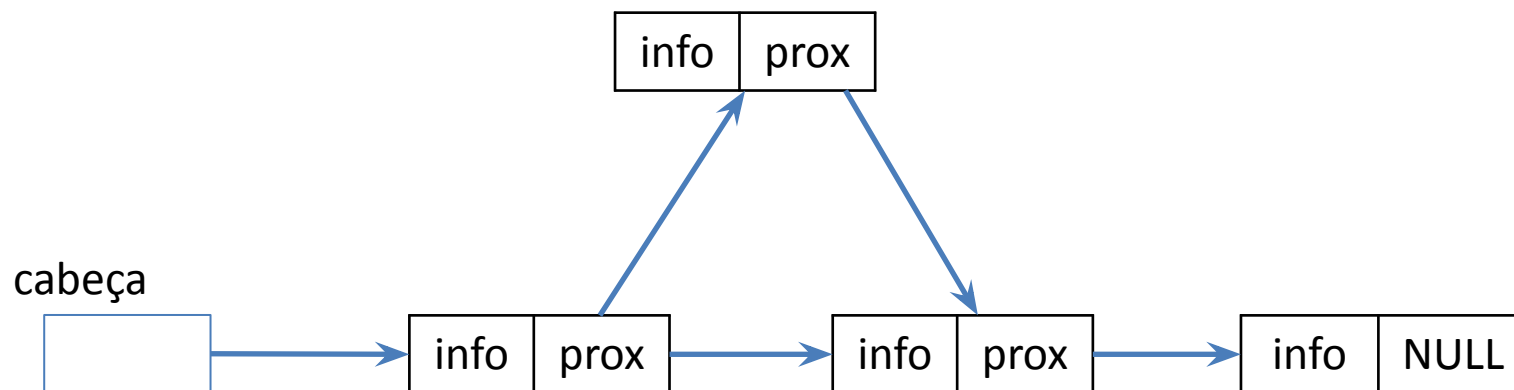
Inserção na última posição



Inserção em posição específica

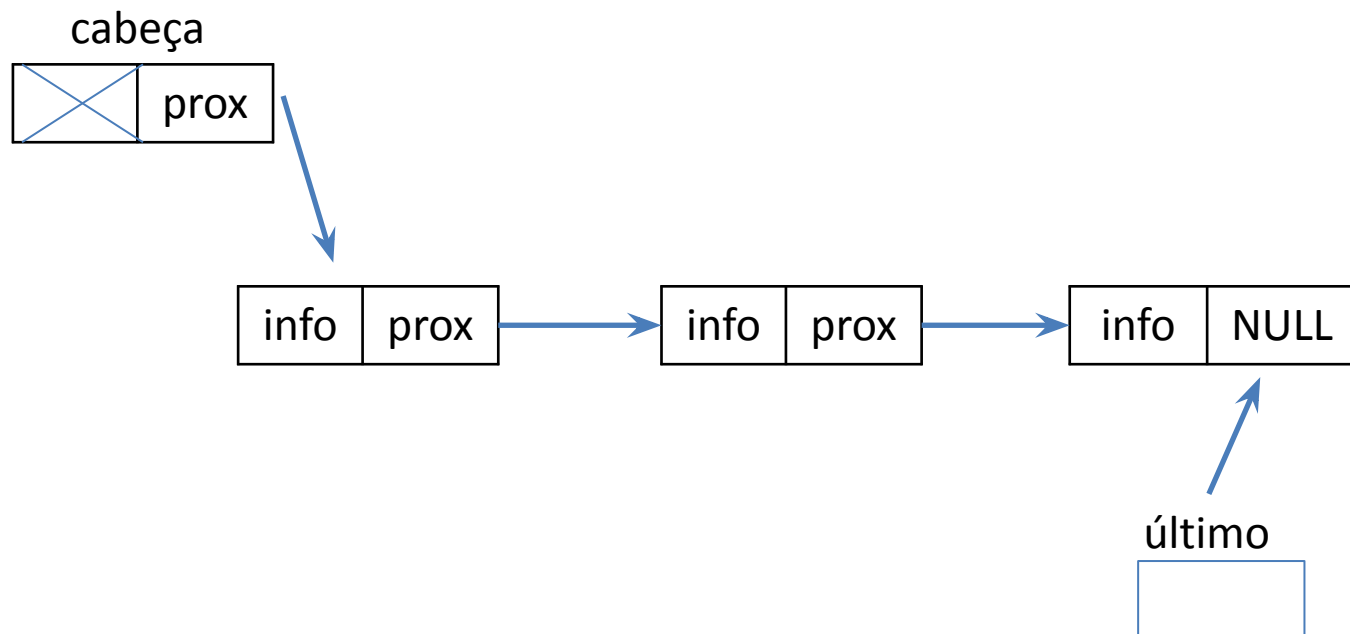


Sistemas para Internet
UFSM



Retirar elemento da lista

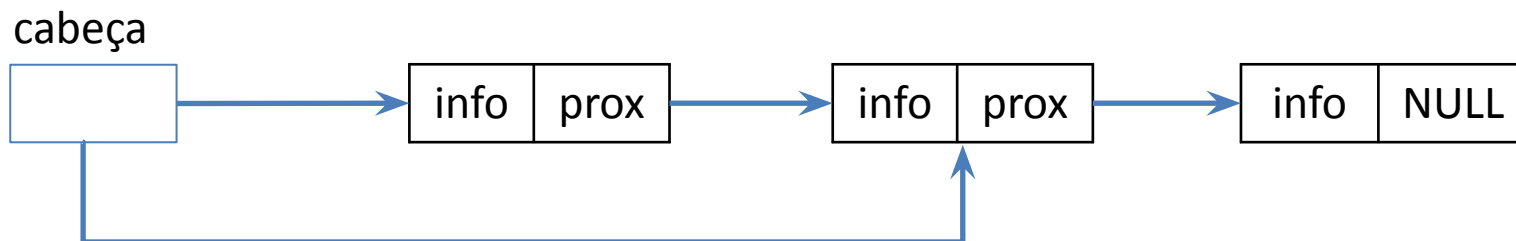
- 3 opções para retirada de elementos:
 - 1ª. posição;
 - Última posição;
 - Posição específica.



Retirar o 1º elemento da lista



Sistemas para Internet
UFSM

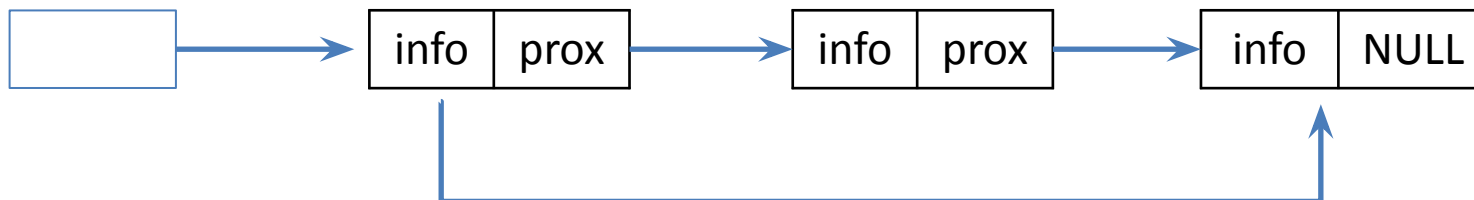


Retirar elemento específico



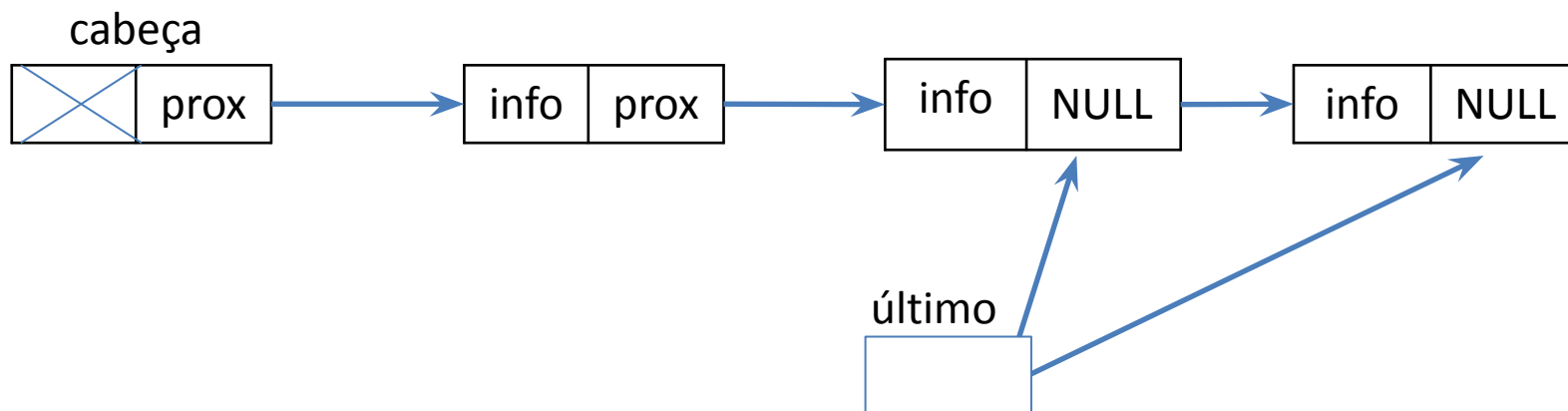
Sistemas para Internet
UFSM

cabeça





Retirar último elemento





Estrutura da lista encadeada

```
12 struct lista {  
13     int info;  
14     struct lista *prox;  
15 };  
16  
17 typedef struct lista Lista;  
18  
19 // FUNÇÃO PARA INSERIR NO INÍCIO  
20 Lista *insere(Lista *l, int i) {  
21     Lista *novo;  
22     novo = (Lista *) malloc (sizeof(Lista));  
23     novo->info = i;  
24     novo->prox = l;  
25     return novo;  
26 }
```

Lista encadeada
deve iniciar
vazia!!

```
61 int main() {  
62     Lista *l;  
63     l = NULL;  
64     A l = insere(l, 43);  
65     B l = insere(l, 75);  
66     C l = insere(l, 30);  
67     D l = insere(l, 12);  
68     imprime(l);  
69     l = retira(l, 75);  
70     printf("\n");  
71     imprime(l);  
72  
73     return 0;  
74 }
```

Símbolo ->: Para acessar os
campos da struct

Operações sobre listas encadeadas



Sistemas para Internet
UFSM

- Inserir no início da lista e imprimir:

```
19 // FUNÇÃO PARA INSERIR NO INÍCIO
20 Lista *insere(Lista *l, int i) {
21     Lista *novo;
22     novo = (Lista *) malloc (sizeof(Lista));
23     novo->info = i;
24     novo->prox = l;
25     return novo;
26 }
27
28 //FUNÇÃO PARA IMPRIMIR ELEMENTOS DA LISTA
29 void imprime (Lista *l)
30 {
31     Lista *p;
32     for (p=l; p!=NULL; p=p->prox)
33         printf("%d ", p->info);
34 }
```

```
61 int main() {
62     Lista *l;
63     l = NULL;
64     l = insere(l, 43);
65     l = insere(l, 75);
66     l = insere(l, 30);
67     l = insere(l, 12);
68     imprime(l);
69     l = retira(l, 75);
70     printf("\n");
71     imprime(l);
72
73     return 0;
74 }
```

Operações sobre listas encadeadas:

Retirar elementos



Sistemas para Internet
UFSM

```
36 //FUNÇÃO PARA RETIRAR UM VALOR DA LISTA
37 Lista *retira(Lista *l, int v) {
38     Lista *ant = NULL;
39     Lista *p = l;
40
41     while (p != NULL && p->info != v) {
42         ant = p;
43         p = p->prox;
44     }
45
46     if (p == NULL){
47         return l;
48     }
49
50     if (ant == NULL) {
51         l = p->prox;
52     }
53     else {
54         ant->prox = p->prox;
55     }
56
57     free(p);
58     return l;
59 }
```

```
61 int main() {
62     Lista *l;
63     l = NULL;
64     l = insere(l, 43);
65     l = insere(l, 75);
66     l = insere(l, 30);
67     l = insere(l, 12);
68     imprime(l);
69     l = retira(l, 75);
70     printf("\n");
71     imprime(l);
72
73     return 0;
74 }
```

Realize um teste de mesa nesta função

Operações sobre listas encadeadas

- **Vantagens:**

- Permite inserir ou retirar itens do meio da lista a um custo constante (importante quando a lista tem de ser mantida em ordem).
- Bom para aplicações em que não existe previsão sobre o crescimento da lista.

- **Desvantagem:**

- Utilização de memória extra para armazenar os apontadores.

Exercícios

1. Descreva textualmente as etapas para criação de uma lista vazia e a inserção de elementos nela;
2. Desenvolva um algoritmo para criar uma lista encadeada vazia. Depois crie uma função que permita que o inserir elementos nela. Por último, apresente os valores pares da lista;
3. Escreva uma função que receba por parâmetro um ponteiro para o primeiro elemento da lista e retira um elemento do início, do meio e do final da lista. Depois apagar da memória o elemento retirado;
4. Escreva uma função que receba por parâmetro um valor e um ponteiro para o primeiro elemento da lista e insere um nó na última posição;



Exercícios 2

1. Faça um algoritmo com uma função que crie uma cópia de uma lista encadeada. Ou seja, uma nova lista de mesmo tamanho com o conjunto de valores na mesma ordem da primeira;
2. Faça um algoritmo com uma função para verificar se duas listas encadeadas são iguais. Duas listas são consideradas iguais se possuem a mesma sequência de valores;
3. Faça um algoritmo para criar uma lista encadeada e inserir elementos (nós) nela. Uma particularidade que esta lista deve ter é que o último elemento nunca aponta para NULL, mas sim para o primeiro elemento.
 - a. Pesquise sobre listas circulares.